GUEST LECTURE · SCORE, VIT VELLORE

Let’s build an AI app in Java

Building AI-Powered Applications with Java and Spring AI

Ujwal Vanjare

Software Engineer, BNSF Railway

28 October 2026 · 6:00 to 8:00 PM IST

Hi, I’m Ujwal

I’m a software engineer at BNSF Railway, working on cloud and infrastructure: Kubernetes, Kafka, OpenShift and much more.

These days, using AI is a big part of how I build and run these systems.

Connect on LinkedIn

YOUR TURN

Quick poll: type a number in the chat

1

I’ve used ChatGPT or Gemini

2

I’ve called an AI API from my own code

3

I’ve built a whole project with AI in it

Pick the highest one that’s true for you.

Here’s the plan

Learn the ideas, watch them work, then build it yourself.

6:05 TO 6:40

1. Foundations

What AI models are, why they matter, where they’re used, and how they work

6:40 TO 7:10

2. Live demo

We build a Campus Assistant together, in your browser. It talks, remembers, books a lab and reads our handbook.

7:15 TO 7:50

3. Your turn

You code two basic examples in your browser. I show two more you can try later.

Break at 7:10 · Quiz and Q&A from 7:50 · Questions in chat any time

DO THIS NOW · 6:00

Start your Codespace now

1

Open the repo link in the chat

2

Sign in to GitHub

3

Code → Codespaces → Create codespace on main

4

Wait about 5 minutes, until the terminal says **All set**

`github.com/usv240/spring-ai-campus-assistant`

If it asks whether to trust the folder, click yes. No account yet? Make a free one at **github.com/signup** and verify your email.

WHAT · 1

What is an AI model, like ChatGPT?

Think of it as a really, really good autocomplete.

```
The capital of India is New Delhi
```

It reads your text, guesses the most likely next word, adds it, and repeats.

Model

The trained AI: GPT, Gemini, Claude, Llama

Prompt

What you send it

Token

A small piece of a word. You pay per token.

WHAT · 2

What is Spring AI?

A Spring library that lets your Java app use AI models with a few lines of code.

Works with many models

Ollama, OpenAI, Gemini, Claude and more

Does the boring parts

Sending requests, reading replies, turning them into Java objects

Adds the useful extras

Memory, calling your Java code, reading your documents

```
chatClient.prompt().user("Hi!").call().content();
```

WHY

Why should you care?

Why add AI to apps

- People can just ask, in plain language
- It reads and sums up long text in seconds
- It takes over small, repetitive tasks

Why in Java

- Most company backends run on Java and Spring
- Companies use AI models; they don’t build them
- So you already know most of what you need

WHERE

Where is it used today?

Banking
Support chat, reading forms and statements

Healthcare
Summing up patient notes and reports

Online shopping
Smarter search, review summaries, shopping help

Education
Tutors that answer from course material

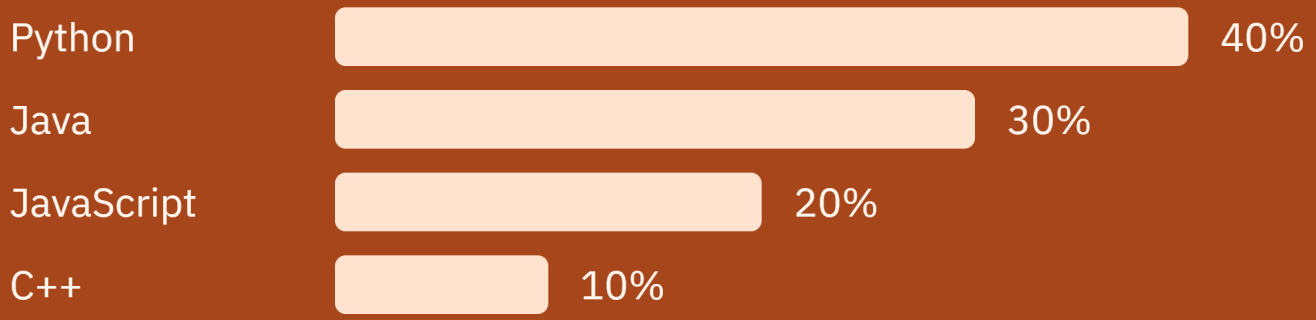
Transport
Searching manuals and logs, answering policy questions

Software teams
Code helpers, writing tests, reviewing code

Most of these are built from the same few pieces: **a chat**, **answers from documents**, and **an assistant that can take actions**. That’s what we’ll build.

HOW · 1 · YOUR TURN

“My favourite programming language is ___”



Low temperature (like 0.1): it almost always picks the top word. Good for facts and code.

High temperature (like 1.0): it takes more chances. Good for stories and ideas.

Made-up numbers, just to show the idea

DO THIS NOW

Start the app now

1

Click inside the terminal

2

Type **mvn spring-boot:run** and press Enter

3

Open **requests.http** and click **Send Request** above the first line, **once only**

The first answer takes about 3 minutes. That’s normal. Don’t click again.

HOW · 2

How it fits into your app



You write the same Java code whichever AI model you use. To switch, you change one dependency and a line of config.

HOW · 3

What it can’t do on its own

Each of these is one of today’s demos.

The problem	What it looks like	We fix it in
It forgets	You tell it your name, and next message it’s gone	Demo 3
It can’t do things	It can explain how to book a lab, but can’t book one	Demo 3
It doesn’t know your data	It has never read our college handbook	Demo 4
It makes things up	Sounds very sure, still wrong	Demo 4
It only gives text	Hard to use the answer inside your code	Demo 2

FOUNDATIONS · PROMPTS

Talk to it like a new teammate, not a search box

Vague

“Write about Java streams.”

Clear

“You’re helping second-year students prepare for a viva. Explain Java streams in 5 short bullet points, with one 3-line code example. If you’re not sure about something, say so.”

Who it’s talking to

What you want, exactly

Format of the answer

What to do when unsure

FOUNDATIONS · REMEMBERING A CONVERSATION

The model forgets everything

It’s like calling customer care and explaining your problem from the start, every single time.

Without memory

Me: My reg no is 21BCE1234.

Me: What’s my reg no?

AI: Sorry, I don’t know that.

With memory

Me: My reg no is 21BCE1234.

Me: What’s my reg no?

AI: It’s 21BCE1234.

The fix is simple: Spring AI keeps the last few messages for each user and sends them along with every new question.

FOUNDATIONS · LETTING AI DO TASKS (“TOOL CALLING”)

Think of a waiter and a kitchen

You order. “Book the AI Lab at 2 PM.”

→

The waiter (AI) writes it down: bookSlot(AI Lab, 14:00)

→

The kitchen (your Java code) actually does it.

→

The waiter brings back the result: “Booked!”

The AI never goes into the kitchen. It only asks. Your code decides what actually happens.

FOUNDATIONS · ANSWERING FROM YOUR DOCUMENTS (“RAG”)

An open-book exam for the AI

Instead of hoping the model already knows, we hand it the right page just before it answers.

Once, when the app starts

1. Read the handbook
2. Cut it into small pieces
3. Turn each piece into numbers that capture its meaning
4. Store them in a vector store

Every time someone asks

1. Find the pieces closest in meaning to the question
2. Paste them into the prompt
3. Let the model answer from them

FOUNDATIONS · WHAT'S COMING NEXT

MCP: one plug for all AI tools

Remember when every phone had a different charger? Then USB-C came along. MCP is that, for AI tools.

The idea

You write a tool once. Then Claude Desktop, coding assistants, and your own apps can all use it.

In Spring AI

```
@McpTool(description = "Free lab slots")
public List<String> freeSlots(String lab)
```

Things I check before anything goes live

This is the part that actually takes time at work.

Check	Why
API keys	Never in the code or on GitHub. Bots scan GitHub for them.
Tricky inputs	Users and documents can try to give the AI new instructions.
Cost	Every token costs money. Use the smallest model that works.
Testing	The same question can give different answers. Test with real examples.
Privacy	Know what data you’re sending outside your company.
Watching it	Spring AI can log every prompt and tool call, and send metrics you can chart.

YOUR TURN · QUICK CHECK

Three questions. Answer in the chat.

1. What does an LLM actually do?

A) Searches Google B) Predicts the next word C) Runs your code

2. You want reliable JSON back. Which temperature?

A) 0.2 B) 1.5

3. To switch from Ollama to OpenAI, what changes?

A) Every controller B) One dependency and the config

What we’re running today

No paid account, no API key. We all run it in a Codespace: a free coding computer in your browser.

- Java 21 and Spring Boot 4.1
- Spring AI 2.0
- Ollama, which runs the AI model inside the Codespace
- A small model called llama3.2

```
# run the app
mvn spring-boot:run
```

DEMO 1

Our first AI endpoint

```
@GetMapping("/chat")
public String chat(@RequestParam String message) {
    return chatClient.prompt()
            .user(message)   // what the user asked
            .call()          // send it to the model
            .content();       // give me the reply as text
}
```

That’s it. A normal Spring controller. No HTTP client, no JSON parsing.

ChatController.java

YOUR TURN · DEMO 1

Give me a topic and an audience

Type it in the chat, like **“recursion, for a 10-year-old”** or **“REST APIs, for my grandmother”**.
Same model, very different answers.

```
chatClient.prompt()
    .system(s → s.text("Explain concepts to {level}.")
        .param("level", audience))  // who it’s talking to
    .user("Explain: " + topic)
    .call()
    .content();
```

Bonus: watch the reply stream in word by word, like ChatGPT does. (Code simplified.)

DEMO 2

Getting neat, organised answers

Text is nice to read, but hard to use in code. So instead of a paragraph, we ask for a Java record.

```
record StudyPlan(String subject, int totalHours, List<StudyTask> tasks, List<String> tips) {}

StudyPlan plan = chatClient.prompt()
    .user("Create a 5-day exam revision plan for Data Structures")
    .call()
    .entity(StudyPlan.class);   // a real Java object
```

Now we can save it to a database, check it, or send it to a frontend, like any other object.

StudyPlanController.java (simplified)

DEMO 3

A tool is a Java method plus a description

```
@Tool(description = "Book a free slot in a lab for a student.
                Always call listFreeSlots first.")
public String bookSlot(String lab, String time, String regNo) {
    // check the slot is free, save the booking
}

chatClient.prompt()
    .user(message)
    .tools(labTools)                    // what it's allowed to ask for
    .advisors(memoryFor(user))          // so it remembers this user
    .call().content();
```

Try: "My reg no is 21BCE1234. Which AI Lab slots are free?" then "Book the 2 PM one." The AI checks free slots first, then books.

LabBookingTools.java · AssistantController.java (code simplified for the slide)

YOUR TURN · GUESS FIRST

At Northfield Institute, what’s the minimum attendance to sit for the final exam?

A
65%

B
75%

C
82%

D
90%

Type A, B, C or D. Then we’ll ask the AI the same question.

DEMO 4

Same question, now with the handbook

```
chatClient.prompt()
    .advisors(QuestionAnswerAdvisor.builder(vectorStore).build())
    .user(question)
    .call()
    .content();
```

One extra line, the advisor, and the model now answers from our documents.

Without the handbook

A confident guess

With the handbook

82%, plus the rule for 75 to 82%

RagController.java · AiConfig.java (simplified)

5-MINUTE BREAK (7:10)

Stretch, grab water, ask anything.

After the break, it’s your turn: you’ll code two small examples yourself. Keep your Codespace open.

ONLY IF YOU NEED IT

If your Codespace stopped

1

Open it again from **github.com/codespaces**

2

In the terminal, run **mvn spring-boot:run**

3

Send one request and wait about 3 minutes for the first answer

Your files are saved, so nothing is lost.

HANDS-ON · EXERCISE 1 · 10 MIN

Give it a personality

Open **AiConfig.java** and find **// EXERCISE 1 START HERE**. Change the text in **defaultSystem**, restart, and send **/chat** again.

```
.defaultSystem("""
    You are Campus Assistant. Answer like an excited
    cricket commentator, in under 60 words.
    """)
```

Restart: click the terminal, press **Ctrl+C**, press the **Up arrow**, press **Enter**. Paste your funniest answer in the chat.

HANDS-ON · EXERCISE 2 · 10 MIN

Add one more detail to the answer

Find **// EXERCISE 2 START HERE** in **StudyPlanController.java**. Add **difficulty** to StudyTask, restart, and send **/study-plan**.

```
record StudyTask(String day, String topic, int hours,
                 String resource, String difficulty) {}
```

No parsing code needed. The model fills in the new field by itself. Got an error page? Click Send Request again.

I'LL SHOW YOU · TRY IT ON YOUR OWN

Write your own tool

Find **// EXERCISE 3 START HERE** at the bottom of **LabBookingTools.java** and paste below it. Try it later: exercise 3 in EXERCISES.md.

```
@Tool(description = "Cancel a lab booking and make the slot free again")
public String cancelBooking(@ToolParam(description = "Lab name, e.g. AI Lab") String lab,
                            @ToolParam(description = "Time slot in HH:mm") String time) {
    lab = findLab(lab);
    time = toHHmm(time);
    List<String> slots = freeSlots.get(lab);
    if (slots == null) return "Cancel failed: there is no lab called " + lab + ".";
    if (!slots.contains(time)) slots.add(time);
    System.out.println(" >>> TOOL EXECUTED: cancelled " + lab + " at " + time);
    return "Cancelled " + lab + " at " + time + ". The slot is free again.";
}
```

Then: book a slot, and ask **"Actually, cancel that booking."** Watch the terminal.

I'LL SHOW YOU · TRY IT ON YOUR OWN

Teach it something new

Paste below the line **<!-- EXERCISE 4 ... -->** at the end of **course-handbook.md**. Restart, because the handbook is read at startup.

```
## Canteen
The main canteen is open from 7 AM to 10 PM every day.
The night canteen near the hostels stays open until
2 AM during exam weeks.
```

Ask **/ask**: "Until when is the canteen open during exams?" Then try **/ask/no-rag**.

YOUR TURN · FINAL QUIZ

Fastest correct answer in the chat wins

1. Which feature lets the assistant remember your reg no?

2. Which feature let it actually book the lab?

3. Which feature let it answer from the handbook?

4. Where should your API key go? (Not in the code!)

5. In the waiter analogy, who is the kitchen?

If I were in your place right now

I’d focus on

- Getting really good at Java and Spring Boot first
- Building one AI project end to end
- Putting it online, with a short demo video
- Going to hackathons, even if you lose

Project ideas to steal

- A bot that answers from your own syllabus
- A resume reviewer that gives a score
- A quiz maker for placement prep
- An expense tracker that reads receipts

What to do after today

Try this week

1. Open the repo and click “Open in Codespaces”. It runs in your browser, nothing to install.
2. Play with the interactive playground page
3. Do exercises 3 and 4 from EXERCISES.md. Stuck? See STUDENT-GUIDE.pdf, and the answers in SOLUTIONS.md
4. Add one thing: a library tool, your own syllabus, or a quiz maker
5. Bonus: give the assistant a goal and watch it plan the steps (end of EXERCISES.md)
6. Put it on GitHub

Good places to learn

- Spring AI docs
- Spring AI for Beginners (free, by Microsoft)
- Official Spring AI examples
- Ollama, to run models for free

Code, exercises and notes: github.com/usv240/spring-ai-campus-assistant

Playground: usv240.github.io/spring-ai-campus-assistant

Q&A

Thank you!

Ask me anything, about today, Java, interviews or hackathons.

Let’s connect on LinkedIn



Scan to connect

Ujwal Vanjare

linkedin.com/in/ujwalvanjare · ujwalvanjare.com

Everything from today: github.com/usv240/spring-ai-campus-assistant